

Modeling Dependable Product-Families: from Use Cases to State Machine Models

Leydi Erazo, Eliane Martins

Institute of Computing
University of Campinas
Campinas, Brazil

leydi.paruma@students.ic.unicamp.br,
eliane@ic.unicamp.br

Juliana Galvani Greghi

Computer Science Department
University of Lavras
Lavras, Brazil
juliana@dcc.ufla.br

Abstract— Product Families are gaining interest because of the increasing demand for customizable products. However, testing a Product Family is a difficult task, in special, for dependable products, in which the exception handling must also be well tested. Model-based testing (MBT) can be useful for testing Product Families, in which a behavior model can be obtained from the requirements; this model being used for automatic test case generation. The use of MBT, however, requires a good modeling expertise from practitioners, as they are mainly used with informal, textual descriptions of the requirements. In this paper, we show how to apply a use-case modeling approach that allows the automatic extraction of a state machine model for a product. Use cases are described in a restricted format that comprises not only the representation of commonalities and variabilities but also the specification of exception handling for fault-tolerant behavior.

Keywords—use case; state machine; product family; exception handling

I. INTRODUCTION

The term Product Family (PF) or Software Product Line (SPL) refers to a set of similar software products that share a common set of features. These products are developed from a common set of core assets in a prescribed way [1]. The development of a PF presents some new challenges for Software Engineering. First, it was necessary to define two processes: *Domain Engineering (DE)* and *Application Engineering (AE)*. The former is concerned with the definition of common and variable parts, as well as the design of an adaptable architecture and the development of reusable components. The *AE*, on the other hand, is concerned with the development of an individual product. The user selects the required features that were defined during the *DE* process. Based on the selected features, the architecture is tailored for the product, and the necessary reusable components are selected. Another challenge is relative to testing: how to be certain that requirements are satisfied by a given new product? [2] Since there is a high number of potential products in a PF, there is also a need to automate product testing.

Model-based testing (MBT) can be very useful for PF testing, in which that MBT provides a technique for generation of test cases using models extracted from software artifacts [3]. One key advantage of MBT is that test

cases can be systematically generated from requirements represented by the model, in that way, automating test case design. A key element for the use of a MBT approach is the model used for software behavior description (e.g. finite state machines). MBT requires solid modeling expertise from developers in order to build models that can be processed by a tool. Since our goal is on test models from use cases representing product-specific behavior, the question is how to allow users that are not familiar with modeling to derive test models for a product that takes into account the exception handling mechanisms?

Use cases (UCs) are used to elicit and document customer requirements. Deriving Analysis Models from UCs is traditionally a manual process, guided by some heuristics [4]. However, there are some proposals to help the automatic model derivation from use cases, such as class models [5] or state machine models [6][7], but they consider only the development of single systems and do not take into account explicitly the exception handling mechanisms.

In this paper, we present an approach to extract state machine models, more specifically, Extended Finite State Machines (EFSMs), representing product-specific behavior. The extraction is based on use case descriptions developed during the Requirements phase. These use cases must be described using a structured language notation. We also added exceptional and variation flows; the former are used to handle exceptional behavior in the software systems whereas the variation flows are used to handle variability when specifying a PF. As a proof-of-concept, we are implementing a prototyping tool, MARITACA, which applies natural language processing techniques to extract the initial version of a product behavior model.

The remainder of the paper is organized as follows. Section II presents some related works. Section III shows the artifacts used in our testing-oriented modeling approach. Section IV approach to generate state machine from use case descriptions, followed by a brief description of the tool in Section V. Finally, Section VI concludes the paper.

II. RELATED WORK

The generation of state machines from use cases has been researched in the last years, for single products as well as for product families. For the sake of space, we only present here

some of the works that inspired in some way the development of our approach. One of the works that provided the basis of ours was Yue et al. that aimed at state-model extraction for single products. They extract various Analysis models, such as class [8], activity and sequence models [9], as well as, state machines [6], from a structured notation for use case description called RUCM (Restricted Use Case Modeling). The use of a restricted notation aimed at reducing ambiguities and facilitates the processing by tools. Besides the structured language and a use case template, their work also proposes a set of transformation rules and algorithms to automatically derive preliminary Analysis models. The authors did an extensive literature review in order to define their notation, and also, they perform experimental studies to assess the usefulness of the approach for end users [10]. The approach is also supported by a tool, aToucan [8].

In what concerns the testing of Product Families, several authors also proposed techniques to derive test cases from use case descriptions modified to cope with variability. The survey presented by [11] presents some of these works. As pointed out in the aforementioned reference, there are many challenges for product-family testing, among them, how to deal with variability in the test artifacts. In this respect, we got inspiration from the FeDRE (Feature-Driven Requirements Engineering) approach for requirements specification of PF [12]. Although this approach is only concerned with the requirements phase, and not with testing, it provides guidelines that establish traceability links among features, products and requirements, which is useful for developers as well as for testers. Our contribution to the PF testing domain is the automatic generation of a product test model that is useful for test case generation.

III. TEST ARTEFACTS

Our goal is a model-based test case generation method for product-specific testing. The method is based mostly on Unified Modeling Language (UML) notations, given their popularity in practice. More specifically, it uses, from UML: use cases, class diagrams, and state machine models. In addition, the method also uses non-UML models to specify variability. One of such models is the **Feature model (FM)** [13], which is used to represent common and variable features in SPLs. The FM is a diagram showing the features, their relationships as well as the constraints among them. The **Product map (PM)** and **Traceability matrix (TM)** [12] were also adopted in our method. The PM relates products to features, indicating which features are used by the products that are foreseen. The TM relates features to use cases. It is not only useful during the development of a new product, as it shows which use cases are required by which features, but also helps when requirements evolve, as each change in the FM will be traced to the use cases and vice-versa. In the following, we describe in more details the Use Case modeling, as it is key for state machine model extraction.

In UML a Use Case Model is composed by a Use Case Diagram (UCD) and a Use Case textual specification (UCTS) [14]. Use cases describe an external view of the system, i.e., they show how the system interacts with its

actors, which can represent people, external systems or hardware devices [4]. When Use Cases are utilized to model PF, it is common to extend them to cope with variability. In the UCD, the Use Cases can be labeled with stereotypes such as [15]: <<kernel>> for those that are common to all products of the PF; <<optional>> for those required for some products but not all, and <<alternative>> when different versions of a use case are required by different products.

The UCD is complemented with the UCTS, which describes each Use Case (UC) in the diagram in a textual form. UML provides no specific notation for specifying UCTS, so it is common practice to adopt some template. The one we adopt is based on RUCM approach [10] because it uses a restricted format to write UCTS in order to facilitate automated UC analysis necessary to extract the behavior models. RUCM was intended for a single system, thus, we extended it to cope with variability. We also made explicit the specification of exceptional behavior [16].

Fig. 1 shows an example of a use case documented according to this structured notation. The example describes the *Open the Door* use case of a Microwave oven system [15]. The fields: brief description, precondition, primary and secondary actors, dependency and generalization are not further explained here, for the sake of space; they are common to many templates and the description given in the table is enough to understand their meaning. The fields *Use Case Type* and *Associated Feature* are intended to handle variability when specifying a PF. The *Use Case Type* specifies whether the use case is kernel (mandatory), optional, or alternative [15]. The *Associated Feature* field specifies the name of the feature(s) associated with the use case. The *Feature Cardinality* specifies, (for alternative features) whether it can have one or more variation points in the use case. Variability can also be specified inside a UC, using variation points, which identify locations in which variation occurs. For the moment, this is the kind of variability considered by the use of variation flows. In the remainder of this section, we focus the discussion on the description of the use case flows, as they are essential for model extraction.

Getting inspiration from [7], we describe a UC as a tuple <Header, Pre, Flows> in which Header was described above. **Pre** is a set of pre-conditions and **Flows** refer to the set of UC's flows. Each flow in **Flows** is also represented by a tuple <Type, Steps, Post>. **Type** contains the type of the flow, which can be one of the following: (i) *basic* (or main) flow represents the “normal flow” of steps for a use case. A use case can only have one basic flow; (ii) *alternative flow*, representing conditional steps that are performed in place of one or more steps in another flow (designated as *reference flow*, or RF), after which the UC continues to be executed; (iii) *exceptional flow*, representing error conditions, and are further divided into recoverable and failure flows [16]. Both flows specify what the system does when a failure occurs in a RF step. The difference between them is that after a recoverable flow execution continues, whereas in a failure flow execution is aborted; (iv) *variant* flows specify steps executed for selected features of a PF. Non-basic flows can also be categorized as *specific* (refers to a single step in a

RF), *bounded* (refers to more than a step, consecutive or not, of the RF) and *global* (refers to all steps of the RF) [5]. Each flow specification ends with a set of post-conditions (**Post**) that must hold when the flows terminate.

Steps represent a set of steps in the flow and are also represented by a tuple <**SNumber**, **Cond**, **Oper**>. **SNumber** is the step number. The steps in a flow are numbered sequentially; this implies that each step is completed before the next one is started. **Cond** is a set of conditions that must hold for the step to be possible; it may be empty. Conditions are expressed using RUCM keywords: **IF-THEN-ELSE-ELSEIF-ENDIF** or **VALIDATES THAT**. In order to consider variability when specifying a PF, we further introduced the keyword **IF-OPTION-THEN-ELSE-ELSEIF-ENDIF**, to specify an alternative or optional feature in a UC that must be selected for a given variation flow to be considered. Fig.1, line 35 shows an example of utilization of this keyword. **Oper** is an action which describes an interaction between: (i) the primary actor and the system (e.g. Fig. 1, line 13); (ii) the system with itself (e.g., modifying its internal state, as in Fig. 1, line 14) or (iii) the system with an actor (e.g., sending a result to an actor). RUCM proposes restriction rules on the use of natural language, such as: actions should be written in the present tense; verbs should be in active voice, among others [5]. In order to ease the processing by a tool, we also add that actions should have the form “The <actor> does something” or “The system does something”.

The exception flows are triggered when there is a violation of the condition associated with the **VALIDATE THAT** keyword. *Recoverable* flows terminate with a **RESUME STEP** keyword. This indicates that the exception flow goes back to the reference flow, meaning that exception handling is well succeeded. A *failure* flow, on the other hand, terminates with an **ABORT** keyword indicating that the system execution can no longer continue.

IV. APPROACH TO GENERATE STATE MACHINE FROM USE CASE DESCRIPTION

Our approach aims to generate a state-machine model representing sequential dependencies among UCs [6][2]. In general, these dependencies are not explicitly identified in textual descriptions. They can be made explicit using pre- and post-conditions. The model to derive is a subset of UML in the form of an extended finite state machine, which can be defined as $M = \langle V, \text{initial}, \text{start}, \text{final}, I, O, CV, T \rangle$. **V** is a finite set of vertices, with **initial** and **final** representing the initial pseudo-states and final state respectively. The **start** state is proposed by Yue et al. [8] to represent the starting state. Except for **initial** and **final**, all the other states are represented by invariants, which are the pre- and post-conditions of the UC. Two states with the same invariants are considered identical. **I** is a finite set of input events, which are represented by UC, as those generally are caused by a request from an actor. **O** represents a finite set of output actions, representing either an included or extended use case or else the outputs generated from the system to an actor. **CV** represents a finite set of context variables used in state invariants or in guards. **T** represents a finite set of transitions.

By default, there is an initial transition from the initial pseudo-state to the start state, as UML requires that every state diagram should have such a transition. This transition is not triggered by an event, but can have an action associated with it. There is also a final transition, to the final state. The other transitions represent possible flows among the UCs.

1	Use Case Id: UC001
2	Use Case Name: Open the door.
3	Use Case Type: Mandatory.
4	Cardinality feature: 1..1
5	Brief Description: The user opens the door.
6	Associate feature: Microwave oven kernel, Light, Turntable.
7	Primary Actor: User.
8	Secondary Actor: None.
9	Pre-condition: The door is closed AND the food is not inserted OR The door is closed AND the food is inserted AND the time is not zero AND the start button is not pressed
10	Dependency: None.
11	Generalization: None.
12	Basic Flow Steps (BFS):
13	1. The user opens the door.
14	2. The system sets the door TO open.
15	3. IF the door is closed AND the food is not inserted THEN
16	4. The system sets the time TO zero.
17	5. The system sets the start button TO not pressed.
18	Postcondition: The door is open AND the food is not inserted AND the time is zero AND the start button is not pressed.
19	Specific Alternative Flow (SAF-1): BFS 2
20	1. ELSEIF the door is closed AND the food is inserted AND the time is zero AND the start button is not pressed THEN
21	2. The system sets the door to open.
22	3. ENDIF
23	Postcondition: The door is open AND the food is inserted AND the time is zero AND the start button is not pressed.
24	Bounded Alternative Flow (BAF):
25	Postcondition:
26	Global Alternative Flow (GAF):
27	Postcondition:
28	Specific Exceptional Flow (SEF):
29	Postcondition:
30	Bounded Exceptional Flow (BEF):
31	Postcondition:
32	Global Exceptional Flow (GEF):
33	Postcondition:
34	Specific Variation Flow (SVF-1): BFS 1
35	1. IF the light OPTION is selected THEN
36	2. The system sets the lamp to on.
37	3. ENDIF
38	Postcondition: The lamp is switched on.
39	Bounded Variation Flow (BVF):
40	Postcondition:
41	Global Variation Flow (GVF):
42	Postcondition:

Fig. 1. Use case *Open the door*

Since states represent pre- and post- conditions, there are as many post-conditions (states) and transitions as flows for a given UC. The guards differentiate the transitions from the origin to a destination state for a given UC. Guards correspond to the **Cond** set that precedes each non-basic flow. In that way, exception flows are represented by a transition whose guard corresponds to the case in which the condition specified with the **VALIDATE THAT** keyword is not satisfied, combined with any other conditions in the flow. The destination state of the transition(s) corresponds to the exception flow post-condition(s). An **ABORT** leads to a final state. For a PF, it is necessary to consider the variation flows. Variation flows are characterized by the **IF-OPTION-THEN-ENDIF** keyword in a flow of a UC, that we designate here as UCv. When the feature (indicated after the **OPTION** word) is selected for the product being modeled, a new state is created with the variation flow's post-condition. Then, a new transition is added from the origin state of UCv to the newly created states. The guard of this transition corresponds to the step's Cond set.

It is worth noting that the extracted state machine contains only simple states, i.e., it has no regions and no

submachine states as allowed in UML state diagrams. States do not have internal activities either, such as entry or exit actions. Furthermore, the state machine is highly dependent on the way use cases are written. Hence, the model may contain redundant or extra states, missing states or transitions as well as incorrect states and transitions [8]. Besides, states and transitions are labeled with natural language sentences. In summary, a refinement phase is necessary to convert the extracted machine to a format understandable by a tool. Despite all, the extracted model is a valuable first step for those not used with state machine modeling.

V. SUPPORTING TOOL

In order to show the feasibility of the approach, we are implementing a prototype tool, MARITACA (state Machine geneRation from TextuAl use Cases). Based on our previous experience with TXT2SMM prototype [17], which extracts state machine models from a narrative text, MARITACA applies Natural Language Processing (NLP) techniques, more specifically, a Part-of-Speech tagger, which processes the words in a text and attaches to each a tag according to its grammatical category (noun, adjectives, verbs and so on). The tagged text is used to extract patterns, that is, word sequences that can be assigned to elements of the state machine: triggers, guards and variables. The assignment of the extracted patterns to these elements is based not only on grammatical information but also on the template structure. Moreover, the MARITACA tool, using the information available in the traceability matrix [12], let users select the desired features for a given product. Then the tool selects the use cases to analyze based on this information. A sketch of the initial extraction from MARITACA is presented in Fig 2.

```

source state: The door is closed AND the food is not inserted
event: Open the door
guard: The door is closed AND the food is not inserted
action: The system sets the door TO open AND The system sets the time TO
zero AND The system sets the start button TO not pressed AND The system
sets the lamp to on
target state: The door is open AND the food is not inserted AND the time
is zero AND the start button is not pressed AND The lamp is switched on

```

Fig. 2. Initial extraction: Use case Open the door

VI. CONCLUSIONS AND FUTURE WORK

We present the generation of state machine models from use cases in the context of model-based test case generation for a PF. The state machines are product-specific and represent control dependencies among UCs. Our work aims at helping not only testers but also domain and requirements analysts in the specification of PFs. A prototype tool is under development, to support the approach. The tool uses a set of rules and natural language processing techniques to extract the state machine from the restricted description of UCs. This paper presents an ongoing work, and hence, improvements to the tool are envisaged, as well as the application of the method to real world product families.

ACKNOWLEDGMENT

The authors would like to thank the CAPES for the support. They are also grateful to the European DEVASSES Project.

REFERENCES

- [1] P. Clements and L. Northrop, *Software product lines: practices and patterns*. Boston, MA: Addison-Wesley Professional, 2002.
- [2] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jezequel, "A requirement-based approach to test product families," in *Software Product-Family Engineering*, Springer Berlin, 2003, pp. 198–210.
- [3] S. Dalal, A. Jain, N. Karunanithi, J. M. Leaton, C. Lott, G. Patton, and B. Horowitz, "Model-Based Testing in Practice," in *Proceedings of the 21st International Conference on Software Engineering*, Los Angeles, California: ACM, 1999, pp. 285–294.
- [4] I. Jacobson, *Object Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley Professional, 1993.
- [5] T. Yue, L. C. Briand, and Y. Labiche, "Facilitating the Transition from Use Case Models to Analysis Models: Approach and Experiments," *ACM Trans. Softw. Eng. Methodol.*, pp. 1–38, 2013.
- [6] T. Yue, S. Ali, and L. Briand, "Automated transition from use cases to UML state machines to support state-based testing," *Proc. 7th Eur. Conf. Model. Found. Appl.*, pp. 115–131, 2011.
- [7] S. Somé, "Beyond Scenarios: Generating State Models from Use Cases," *ICSE 2002 Work. Scenar. state Mach. Model. algorithms, tools.*, pp. 456–462, 2002.
- [8] T. Yue, L. C. Briand, and Y. Labiche, "aToucan: An automated framework to derive UML analysis models from use case models," *ACM Trans. Softw. Eng. Methodol.*, 2015.
- [9] T. Yue, L. C. Briand, and Y. Labiche, "An Automated Approach to Transform Use Cases into Activity Diagrams," *Proc. 6th Eur. Conf. Model. Found. Appl.*, pp. 337–353, 2010.
- [10] T. Yue, L. C. Briand, and Y. Labiche, "A Use Case Modeling Approach to Facilitate the Transition towards Analysis Models :," in *International Conference on Model Driven Engineering Languages and Systems*, Springer Berlin, 2009, pp. 484–498.
- [11] B. P. Lamancha, M. P. Usaola, and M. P. Velthuis, "Software product line testing - A Systematic Review," *ICSOF (1)*, pp. 23–30, 2009.
- [12] R. P. De Oliveira, E. Insfran, S. Abrahao, J. Gonzalez-Huerta, D. Blanes, S. Cohen, and E. S. De Almeida, "A feature-driven requirements engineering approach for software Product Lines," in *Software Components, Architectures and Reuse (SBCARs), 2013 VII Brazilian Symposium on.*, 2013, pp. 1–10.
- [13] K. Kang, S. Cohen, J. Hess, W. Novak, Peterson, and Spencer, "Feature-Oriented Domain Analysis (FODA) Feasibility Study," 1990.
- [14] The Object Management Group, "Unified Modeling Language: Superstructure Version 2.0," 2004.
- [15] H. Gomma, *Designing software product line with UML: From Use Cases to Pattern-Based Software Architectures*. Addison-Wesley, 2004.
- [16] C. M. F. Rubira, R. de Lemos, G. R. M. Ferreira, and C. F. Filho, "Explicit Representation of Exception Handling in the Development of Dependable Component-Based Systems," *Softw. Pract. Exp.*, pp. 195–236, 2005.
- [17] J. G. Gregghi, E. Martins, and A. M. B. R. Carvalho, "Semi-automatic Generation of Extended Finite State Machines from Natural Language Standard Documents," in *Dependable Systems and Networks Workshops (DSN-W)*, 2015, pp. 45–50.